

DATA STRUCTURES AND ALGORITHMS

LECTURE 3

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

- Dynamic Array
- Iterators

- Containers
 - ADT Bag
 - ADT SortedBag
 - ADT Set
 - ADT SortedSet
 - ADT Matrix
 - ADT Map
 - ADT SortedMap
 - ADT MultiMap
 - ADT SortedMultimap
 - ADT Stack

- There are many different containers, based on different properties:
 - do the elements have to be unique?
 - do the elements have positions assigned?
 - can we access any element or just some specific ones?
 - do we have simple elements, or key-value pairs?

- The ADT Bag is a container in which the elements are not unique and they do not have positions.
- Interface of the Bag was discussed at Seminar 1.

ADT Bag - representation

- A Bag can be represented using several data structures, one of them being a dynamic array (others will be discussed later)
- Independently of the chosen data structure, there are two options for storing the elements:
 - Store separately every element that was added (R1).
 - Store each element only once and keep a frequency count for it (R2).

ADT Bag - R1 example

- Assume a dynamic array as data structure for the representation (but the idea is applicable for other representations as well)
- Assume that we have a Bag with the following numbers: 4, 1, 6, 4, 7, 2, 1, 1, 1, 9
- In R1 the Bag looks in the following way:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
4	1	6	4	7	2	1	1	1	9						

- Add element -5

ADT Bag - R1 example

- Assume a dynamic array as data structure for the representation (but the idea is applicable for other representations as well)
- Assume that we have a Bag with the following numbers: 4, 1, 6, 4, 7, 2, 1, 1, 1, 9
- In R1 the Bag looks in the following way:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
4	1	6	4	7	2	1	1	1	9						

- Add element -5

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
4	1	6	4	7	2	1	1	1	9	-5					

ADT Bag - R1 example

- Remove element 6

ADT Bag - R1 example

- Remove element 6

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
4	1	-5	4	7	2	1	1	1	9						

- Obs.** The order of the elements in a Bag is not important, you do not need to keep the same order in which they were added. So, when an element needs to be removed, you do not need to do the array-specific *move each element one position to the left* operation, you can simply take the last element and put it in the place of the removed one, for a small improvement in the number of operations.
- What is the complexity of the remove operation when:
 - we move elements to the left ?
 - we take the last element and put it to replace the removed one?

ADT Bag - R2 example

- Assume the same elements as before: 4, 1, 6, 4, 7, 2, 1, 1, 1, 9
- In R2 the Bag looks in the following way:

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9			
freq	2	4	1	1	1	1			

- Add element -5

ADT Bag - R2 example

- Assume the same elements as before: 4, 1, 6, 4, 7, 2, 1, 1, 1, 9
- In R2 the Bag looks in the following way:

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9			
freq	2	4	1	1	1	1			

- Add element -5

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9	-5		
freq	2	4	1	1	1	1	1		

- Add element 7

ADT Bag - R2 example

- Assume the same elements as before: 4, 1, 6, 4, 7, 2, 1, 1, 1, 9
- In R2 the Bag looks in the following way:

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9			
freq	2	4	1	1	1	1			

- Add element -5

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9	-5		
freq	2	4	1	1	1	1	1		

- Add element 7

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9	-5		
freq	2	4	1	2	1	1	1		

ADT Bag - R2 example

- Remove element 6

ADT Bag - R2 example

- Remove element 6

	1	2	3	4	5	6	7	8	9
elems	4	1	-5	7	2	9			
freq	2	4	1	2	1	1			

- Remove element 1

ADT Bag - R2 example

- Remove element 6

	1	2	3	4	5	6	7	8	9
elems	4	1	-5	7	2	9			
freq	2	4	1	2	1	1			

- Remove element 1

	1	2	3	4	5	6	7	8	9
elems	4	1	-5	7	2	9			
freq	2	3	1	2	1	1			

ADT Bag - Dynamic Array specific representations

- Besides the two representations presented above which can be used for other data structures as well, there are two other possible representations which are specific for dynamic arrays.

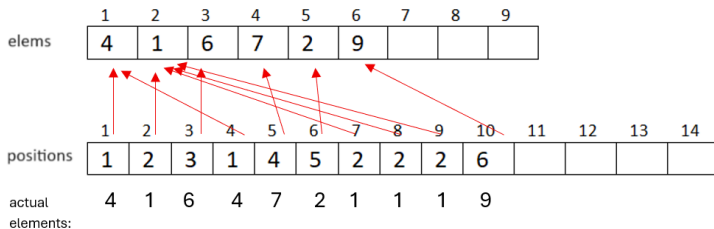
ADT Bag - R3

- Another representation would be to store the unique elements in a dynamic array and store separately the positions from this array for every element that appears in the Bag (R3).
- Assume the same elements as before: 4, 1, 6, 4, 7, 2, 1, 1, 1, 9
- In R3 the Bag looks in the following way (assume 1-based indexing):

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9			

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
positions	1	2	3	1	4	5	2	2	2	6				

ADT Bag - R3 example



ADT Bag - R3 example

- Add element -5

ADT Bag - R3 example

- Add element -5

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9	-5		

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
positions	1	2	3	1	4	5	2	2	2	6	7			

- Add element 7

ADT Bag - R3 example

- Add element -5

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9	-5		

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
positions	1	2	3	1	4	5	2	2	2	6	7			

- Add element 7

	1	2	3	4	5	6	7	8	9
elems	4	1	6	7	2	9	-5		

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
positions	1	2	3	1	4	5	2	2	2	6	7	4		

ADT Bag - R3 example

- Remove element 6

ADT Bag - R3 example

- Remove element 6

	1	2	3	4	5	6	7	8	9
elems	4	1	-5	7	2	9			

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
positions	1	2	4	1	4	5	2	2	2	6	3			

- Removing element 6 implies a few steps:
 - Finding the position of 6 in the *elems* array, let's call it *elempos*.
 - Finding *elempos* in the *positions* array, and removing it (move the last element in its place).
 - Checking if *elempos* still appears in the *positions* array. If not, it means that we have removed the last occurrence of the element from the Bag. We need to remove element 6 from the *elems* array as well.
 - Removing 6 from the *elems* array, by moving the last element in its place.
 - Changing the value of the position for the last element (moved in place of 6) in the *positions* array.

ADT Bag - R3 example

	1	2	3	4	5	6	7	8	9
elems	4	1	-5	7	2	9			

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
positions	1	2	4	1	4	5	2	2	2	6	3			

- Remove element 1

ADT Bag - R3 example

	1	2	3	4	5	6	7	8	9
elems	4	1	-5	7	2	9			

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
positions	1	2	4	1	4	5	2	2	2	6	3			

- Remove element 1

	1	2	3	4	5	6	7	8	9
elems	4	1	-5	7	2	9			

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
positions	1	3	4	1	4	5	2	2	2	6				

- If the elements of the Bag are integer numbers (and a dynamic array is used for storing them), another representation is possible, where the positions of the array represent the elements and the value from the position is the frequency of the element. Thus, the frequency of the minimum element is at position 1 (assume 1-based indexing).
- Assume the same elements as before: 4, 1, 6, 4, 7, 2, 1, 1, 1, 9
- In R4 the Bag looks in the following way:

	1	2	3	4	5	6	7	8	9	10	11
frequencies	4	1	0	2	0	1	1	0	1		

Minimum element: 1

ADT Bag - R4 example

- Add element -5

ADT Bag - R4 example

- Add element -5

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
frequencies	1	0	0	0	0	0	4	1	0	2	0	1	1	0	1		

Minimum element: -5

- When indexing starts from 1, the element in the dynamic array that is on position i , represents the actual value:
 $minimum + i - 1 \Rightarrow$ position of an element e is
 $e - minimum + 1$
- Add element 7

ADT Bag - R4 example

- Add element -5

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
frequencies	1	0	0	0	0	0	4	1	0	2	0	1	1	0	1		

Minimum element: -5

- When indexing starts from 1, the element in the dynamic array that is on position i , represents the actual value:
 $minimum + i - 1 \Rightarrow$ position of an element e is
 $e - minimum + 1$
- Add element 7

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
frequencies	1	0	0	0	0	0	4	1	0	2	0	1	2	0	1		

Minimum element: -5

ADT Bag - R4 example

- Remove element 6

ADT Bag - R4 example

- Remove element 6

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
frequencies	1	0	0	0	0	0	4	1	0	2	0	0	2	0	1		

Minimum element: -5

- Remove element 1

ADT Bag - R4 example

- Remove element 6

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
frequencies	1	0	0	0	0	0	4	1	0	2	0	0	2	0	1		

Minimum element: -5

- Remove element 1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
frequencies	1	0	0	0	0	0	3	1	0	2	0	0	2	0	1		

Minimum element: -5

ADT Bag - R4 example I

- If the elements are far from each other, R4 can be a representation which uses a lot of memory.
- Can you think of an advantage that R4 has (over R1, R2 and R3)?

- Resize is an interesting operation in case of R4: you must resize when you add a new maximum or a new minimum (you have no place where to put them), however, when you remove the last occurrence of the minimum or maximum element, in theory you could choose to not resize (just leave the 0 frequency). However, this is going to possibly waste even more memory, so either you need a more complex mechanism for checking when to resize (for example, when the first/last X% of the array is empty) and during resize you need to make sure to not copy the empty ends of the array. Or you can simply decide to resize whenever the minimum or the maximum is removed.

ADT Bag - R4 Iterator

- How would you define the iterator in case of this representation?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
frequencies	1	0	0	0	0	0	4	1	0	2	0	0	2	0	1		

Minimum element: -5

ADT Bag - R4 Iterator

- How would you define the iterator in case of this representation?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
frequencies	1	0	0	0	0	0	4	1	0	2	0	0	2	0	1		

Minimum element: -5

- Even if the array used for representing the Bag contains *gaps* (positions with zero frequency, representing elements which are not there), the iterator is only allowed to return existing elements.
- This means that when we go to the next element, we need to skip the positions with 0 frequency $\Rightarrow$ complexity of operation *next* will not be $\Theta(1)$.
- Also, remember, since it is a frequency-based representation, iterator needs to consider a *current frequency* as well (discussed in Seminar 1 for R1 and R2, but it applies to every frequency-based representation).

ADT Sorted Bag

- There are no positions in a Bag, but sometimes we need the elements to be sorted $\Rightarrow$ ADT SortedBag.
- Remember, these were the operations in the interface of the ADT Bag:
 - `init(b)`
 - `add(b, e)`
 - `remove(b, e)`
 - `search(b, e)`
 - `nrOfOccurrences(b, e)`
 - `size(b)`
 - `iterator(b, it)`
 - `destroy`
- What should be different (new operations, removed operations, modified operations) in case of a *SortedBag*?

ADT Sorted Bag

- The only modification in the interface is that the `init` operation receives a *relation* as parameter
- Domain of Sorted Bag:
 - $\mathcal{SB} =$
 $\{\mathbf{sb} \mid \mathbf{sb} \text{ is a sorted bag that uses a relation to order the elements}\}$
- `init (sb, rel)`
 - **descr:** creates a new, empty sorted bag, where the elements will be ordered based on a relation
 - **pre:** $rel \in \text{Relation}$
 - **post:** $sb \in \mathcal{SB}$, sb is an empty sorted bag which uses the relation rel

The relation

- Usually there are two approaches, when we want to order elements:
 - Assume that they have a *natural ordering*, and use this ordering (for ex: alphabetical ordering for strings, ascending ordering for numbers, etc.).
 - Sometimes, we want to order the elements in a different way than the natural ordering (or there is no natural ordering) $\Rightarrow$ we use a relation
 - A relation will be considered as a function with two parameters (the two elements that are compared) which returns *true* if they are in the correct order, or *false* if they should be reversed.

ADT Sorted Bag

- The other operations from the interface are the same for a Bag and a SortedBag.
- A SortedBag does not have positions; the way in which the elements are stored internally is hidden. Then how can I see the difference between a Bag and a SortedBag?

ADT Sorted Bag

- The other operations from the interface are the same for a Bag and a SortedBag.
- A SortedBag does not have positions; the way in which the elements are stored internally is hidden. Then how can I see the difference between a Bag and a SortedBag?
 - the iterator for a SortedBag has to return the elements in the order given by the relation.

ADT Sorted Bag

- The other operations from the interface are the same for a Bag and a SortedBag.
- A SortedBag does not have positions; the way in which the elements are stored internally is hidden. Then how can I see the difference between a Bag and a SortedBag?
 - the iterator for a SortedBag has to return the elements in the order given by the relation.
 - Since the iterator operations should have a $\Theta(1)$ complexity, this means that internally the elements have to be stored based on the relation.

ADT SortedBag - representation

- A SortedBag can be represented using several data structures, one of them being the dynamic array (others will be discussed later):
- Independently of the chosen data structure, there are two options for storing the elements (similar to R1 and R2 discussed for a Bag):
 - Store separately every element that was added (in the order given by the relation)
 - Store each element only once (in the order given by the relation) and keep a frequency count for it
- Assuming that the elements are integer numbers and we use a Dynamic array as data structure, is R4 from the Bag a possible representation for a SortedBag?

- Consider the following problem: *in order to avoid electoral fraud (especially the situation when someone votes multiple times in different locations) we want to build a software system which stores the personal numerical code (CNP) of everyone who votes.* What would be the characteristics of the container used to store these personal numerical codes?

- Consider the following problem: *in order to avoid electoral fraud (especially the situation when someone votes multiple times in different locations) we want to build a software system which stores the personal numerical code (CNP) of everyone who votes.* What would be the characteristics of the container used to store these personal numerical codes?
 - The elements have to be unique
 - The order of the elements is not important
- The container in which the elements have to be unique and the order of the elements is not important (there are no positions) is the **ADT Set**.

- Domain of the ADT Set:

$\mathcal{S} = \{s | s \text{ is a set with elements of the type TElem}\}$

- **init (s)**
 - **descr:** creates a new empty set
 - **pre:** true
 - **post:** $s \in \mathcal{S}$, s is an empty set.

- **add**(s, e)
 - **descr:** adds a new element into the set if it is not already in the set
 - **pre:** $s \in \mathcal{S}, e \in TElem$
 - **post:** $s' \in \mathcal{S}, s' = s \cup \{e\}$ (e is added only if it is not in s yet. If s contains the element e already, no change is made).
 $add \leftarrow$ true if e was added to the set, *false* otherwise.

- `remove(s, e)`
 - **descr:** removes an element from the set.
 - **pre:** $s \in \mathcal{S}, e \in TElem$
 - **post:** $s \in \mathcal{S}, s' = s \setminus \{e\}$ (if e is not in s , s is not changed).
 $remove \leftarrow \text{true}$, if e was removed, *false* otherwise

- `search(s, e)`
 - **descr:** verifies if an element is in the set.
 - **pre:** $s \in \mathcal{S}, e \in TElem$
 - **post:**

$$search \leftarrow \begin{cases} True, & \text{if } e \in s \\ False, & \text{otherwise} \end{cases}$$

- **size(s)**
 - **descr:** returns the number of elements from a set
 - **pre:** $s \in \mathcal{S}$
 - **post:** $\text{size} \leftarrow$ the number of elements from s

- `isEmpty(s)`
 - **descr:** verifies if the set is empty
 - **pre:** $s \in \mathcal{S}$
 - **post:**

$$isEmpty \leftarrow \begin{cases} True, & \text{if } s \text{ has no elements} \\ False, & \text{otherwise} \end{cases}$$

- `iterator(s, it)`
 - **descr:** returns an iterator for a set
 - **pre:** $s \in \mathcal{S}$
 - **post:** $it \in \mathcal{I}$, it is an iterator over the set s

- **destroy** (s)
 - **descr:** destroys a set
 - **pre:** $s \in S$
 - **post:** the set s was destroyed.

- Other possible operations (characteristic for sets from mathematics):
 - reunion of two sets
 - intersection of two sets
 - difference of two sets (elements that are present in the first set, but not in the second one)

ADT Set - representation

- If a Dynamic Array is used as data structure and the elements of the set are numbers, we can choose a representation in which the elements are represented by the positions in the dynamic array and a boolean value from that position shows if the element is in the set or not.
- Assume a Set with the following numbers: 4, 2, 10, 7, 6.
- This Set would be represented in the following way (the formulae discussed at Bag R4 can be applied here as well):

1	2	3	4	5	6	7	8	9
T	F	T	F	T	T	F	F	T

Minimum element: 2

ADT Set - representation

- Add element -3

ADT Set - representation

- Add element -3

1	2	3	4	5	6	7	8	9	10	11	12	13	14
T	F	F	F	F	T	F	T	F	T	T	F	F	T

Minimum element: -3

- Remove element 10

ADT Set - representation

- Add element -3

1	2	3	4	5	6	7	8	9	10	11	12	13	14
T	F	F	F	F	T	F	T	F	T	T	F	F	T

Minimum element: -3

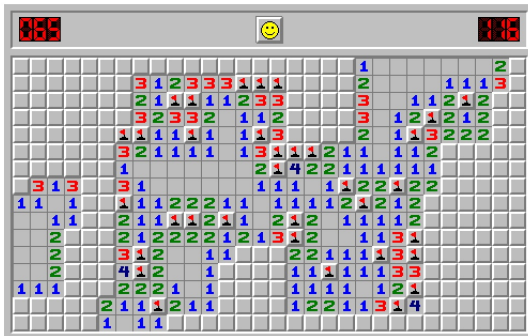
- Remove element 10

1	2	3	4	5	6	7	8	9	10	11
T	F	F	F	F	T	F	T	F	T	T

Minimum element: -3

- We can have a Set where the elements are ordered based on a *relation* $\Rightarrow$ *SortedSet*.
- The only change in the interface is for the *init* operation that will receive the *relation* as parameter.
- For a sorted set, the iterator has to iterate through the elements in the order given by the *relation*, so we need to keep them ordered in the representation.

- Imagine that you wanted to implement this game:



Source: <http://minesweeperonline.com/#>

- What would be the specifics of the container needed to store the game board (location of the mines)?

- The **ADT Matrix** is a container that represents a two-dimensional array.
- Each element has a unique position, determined by two indexes: its line and column.
- The domain of the ADT Matrix: $\mathcal{MAT} = \{mat \mid mat \text{ is a matrix with elements of the type TElem}\}$
- What operations should we have for a Matrix?

- **init**(*mat*, *nrL*, *nrC*)
 - **descr:** creates a new matrix with a given number of lines and columns
 - **pre:** $nrL \in N^*$ and $nrC \in N^*$
 - **post:** $mat \in \mathcal{MAT}$, *mat* is a matrix with *nrL* lines and *nrC* columns
 - **throws:** an exception if *nrL* or *nrC* is negative or zero

- **nrLines(mat)**
 - **descr:** returns the number of lines of the matrix
 - **pre:** $mat \in \mathcal{MAT}$
 - **post:** $nrLines \leftarrow$ returns the number of lines from mat

- **nrCols(mat)**
 - **descr:** returns the number of columns of the matrix
 - **pre:** $mat \in \mathcal{MAT}$
 - **post:** $nrCols \leftarrow$ returns the number of columns from mat

- `element(mat, i, j)`
 - **descr:** returns the element from a given position from the matrix (assume 1-based indexing)
 - **pre:** $mat \in \mathcal{MAT}, 1 \leq i \leq nrLines, 1 \leq j \leq nrColumns$
 - **post:** $element \leftarrow$ the element from line i and column j
 - **throws:** an exception if the position (i, j) is not valid (less than 1 or greater than $nrLines/nrColumns$)

- **modify**(mat, i, j, val)
 - **descr:** sets the element from a given position to a given value (assume 1-based indexing)
 - **pre:** $mat \in \mathcal{MAT}$, $1 \leq i \leq nrLines$, $1 \leq j \leq nrColumns$, $val \in TElem$
 - **post:** the value from position (i, j) is set to val . $modify \leftarrow$ the old value from position (i, j)
 - **throws:** an exception if position (i, j) is not valid (less than 1 or greater than nrLine/nrColumns)

- Other possible operations:
 - get the (first) position of a given element
 - create an iterator that goes through the elements by columns
 - create an iterator the goes through the elements by lines
 - etc.

ADT Matrix - representation

- Usually a sequential representation is used for a Matrix (we memorize all the lines one after the other in a consecutive memory block).
- If this sequential representation is used, for a matrix with N lines and M columns, the element from position (i, j) can be found at the memory address:
address of element from position $(i, j) = \text{address of the matrix} + (i * M + j) * \text{size of an element}$
- The above formula works for 0-based indexing, but can be adapted to 1-based indexing as well.

ADT Matrix - representation

Size of int: 4

Address of matrix (5 rows, 8 cols): 6224024

Address of element 0, 0: 6224024

Address of element 2, 4: 6224104

Address of element 2, 5: 6224108

Address of element 2, 6: 6224112

Address of element 2, 7: 6224116

Address of element 3, 0: 6224120

Address of element 3, 4: 6224136

Address of element 4, 7: 6224180

ADT Matrix - representation

- In the Minesweeper game example above we have a matrix with 480 elements ($16 * 30$) but only 99 bombs.
- If the Matrix contains many values of 0 (or 0_{TElem}), we have a *sparse matrix*, where it is more (space) efficient to memorize only the elements that are different from 0.

Sparse Matrix Example

0	33	0	100	1	0	0	9
2	0	2	0	2	0	7	0
0	4	0	0	3	0	0	0
17	0	0	10	0	16	0	7
0	0	0	0	0	0	0	0
0	1	0	13	0	8	0	29

- Number of lines: 6
- Number of columns: 8
- from 48 positions, 30 values are equal to 0.

- We can memorize (line, column, value) triples, where value is different from 0 (or 0_{TElem}). For efficiency, we memorize the elements sorted by the (line, column) pairs (if the lines are different we order by line, if they are equal we order by column) - R1.
- If a (line, column) pair is not in the representation, we know that its value is 0 (or 0_{TElem}):
- Triples can be stored in a dynamic array or other data structures (that will be discussed later):

Sparse Matrix - R1 example

0	33	0	100	1	0	0	9
2	0	2	0	2	0	7	0
0	4	0	0	3	0	0	0
17	0	0	10	0	16	0	7
0	0	0	0	0	0	0	0
0	1	0	13	0	8	0	29

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

Sparse Matrix - R1 example

- In an ADT Matrix, there is no operation to add an element or to remove an element. In the interface we only have the *modify* operation which changes a value from a position. If we represent the matrix as a sparse matrix, the *modify* operation might add or remove an element to/from the underlying data structure. But the operation from the interface is still called *modify*.

- When we have a Sparse Matrix (i.e., we keep only the values different from 0), for the modify operation we have four different cases, based on the value of the element currently at the given position (let's call it *current_value*) and the new value that we want to put on that position (let's call it *new_value*).
 - $current_value = 0$ and $new_value = 0 \Rightarrow$ do nothing
 - $current_value = 0$ and $new_value \neq 0 \Rightarrow$ insert in the data structure
 - $current_value \neq 0$ and $new_value = 0 \Rightarrow$ remove from the data structure
 - $current_value \neq 0$ and $new_value \neq 0 \Rightarrow$ just change the value in the data structure

Sparse Matrix - R1 example

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (1, 5) to 0

Sparse Matrix - R1 example

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (1, 5) to 0

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Line	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (3, 3) to 19

Sparse Matrix - R1 example

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (1, 5) to 0

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Line	1	1	1	2	2	2	2	3	3	4	4	4	4	6	6	6	6
Col	2	4	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

- Modify the value from position (3, 3) to 19

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Line	1	1	1	2	2	2	2	3	3	3	4	4	4	4	6	6	6	6
Col	2	4	8	1	3	5	7	2	3	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	19	3	17	10	16	7	1	13	8	29

- We can see that in the previous representation there are many consecutive elements which have the same value in the line array. The array containing this information could be compressed, in the following way:
 - Keep the *Col* and *Value* arrays as in the previous representation.
 - For the lines, have an array of number of lines + 1 element, in which at position i we have the position from the *Col* array where the sequence of elements from line i begins.
 - Thus, elements from line i are in the *Col* and *Value* arrays between the positions $[Line[i], Line[i+1])$.
- This is called **compressed sparse line representation**.
- **Obs:** In order for this representation to work, in the *Col* and *Value* arrays the elements have to be stored by rows (first elements of the first row, then elements of second row, etc.)

Sparse Matrix - R2 example

Diagram illustrating the mapping of lines to columns in a 2D array:

	1	2	3	4	5	6	7
Lines	1	5	9	11	15	15	19

Arrows indicate the mapping from the 'Lines' row to the 'Col' row in the 2D array below:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Col	2	4	5	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	1	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

Sparse Matrix - R2 example

- Modify the value from position (1, 5) to 0

Sparse Matrix - R2 example

- Modify the value from position (1, 5) to 0
 - First we look for element on position (1,5).
 - Elements from line 1 are between positions 1 and 4 (inclusive)
 - Since we have there an item with column 5, we found our element
 - Setting to 0, means removing from *Col* and *Value* array.
 - In *Lines* array just the values change, not the size of the array.

	1	2	3	4	5	6	7	
Lines	1	4	8	10	14	14	18	

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Col	2	4	8	1	3	5	7	2	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	3	17	10	16	7	1	13	8	29

Sparse Matrix - R2 example

- Modify the value from position (3, 3) to 19

Sparse Matrix - R2 example

- Modify the value from position (3, 3) to 19
 - First we look for element on position (3,3)
 - Elements from line 3 are between positions 8 and 9 (inclusive)
 - Since we have no column 3 there, at this position currently the value is 0. To set it to 19 we need to insert a new element in the *Col* and *Value* array.
 - In *Lines* array just the values change, not the size of the array

Diagram illustrating the mapping of lines to columns in a 2D array:

	1	2	3	4	5	6	7
Lines	1	4	8	11	15	15	19

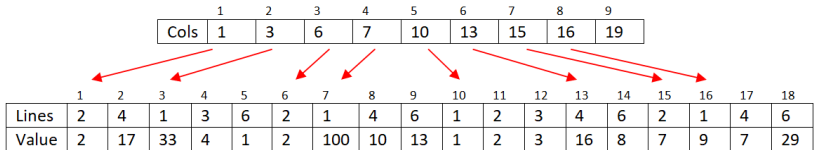
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Col	2	4	8	1	3	5	7	2	3	5	1	4	6	8	2	4	6	8
Value	33	100	9	2	2	2	7	4	19	3	17	10	16	7	1	13	8	29

Red arrows indicate the mapping from the 'Lines' table to the 'Col' and 'Value' rows in the 2D array:

- Line 1 maps to Column 1.
- Line 4 maps to Column 4.
- Line 8 maps to Column 8.
- Line 11 maps to Column 11.
- Line 15 maps to Column 15.
- Line 15 maps to Column 14.
- Line 19 maps to Column 19.

- In a similar manner, we can define **compressed sparse column representation**:
 - We need two arrays *Lines* and *Values* for the non-zero elements, in which first the elements of the first column are stored, then elements from the second column, etc.
 - We need an array with $\text{nrColumns} + 1$ elements, in which at position i we have the position from the *Lines* array where the sequence of elements from column i begins.
 - Thus, elements from column i are in the *Lines* and *Value* arrays between the positions $[\text{Col}[i], \text{Col}[i+1])$.

Sparse Matrix - R3 example



- Consider the following problem: *we have a text and want to find the word that appears most frequently in this text.* How would you do it, and what would be the characteristics of the container used for this problem?

- Consider the following problem: *we have a text and want to find the word that appears most frequently in this text*. How would you do it, and what would be the characteristics of the container used for this problem?
 - We need key (word) - value (number of occurrence) pairs
 - Keys should be unique
 - Order of the keys is not important
- The container in which we store key - value pairs, and where the keys are unique and they are in no particular order is the **ADT Map** (or Dictionary)

- Domain of the ADT Map:

$\mathcal{M} = \{m \mid m \text{ is a map with elements } e = \langle k, v \rangle, \text{ where } k \in T\text{Key} \text{ and } v \in T\text{Value}\}$

ADT Map - Interface I

- **init(m)**
 - **descr:** creates a new empty map
 - **pre:** true
 - **post:** $m \in \mathcal{M}$, m is an empty map.

ADT Map - Interface II

- `destroy(m)`
 - **descr:** destroys a map
 - **pre:** $m \in \mathcal{M}$
 - **post:** m was destroyed

ADT Map - Interface III

- $\text{add}(m, k, v)$
 - **descr:** add a new key-value pair to the map (the operation can be called *put* as well). If the key is already in the map, the corresponding value will be replaced with the new one. The operation returns the old value, or 0_{TValue} if the key was not in the map yet.
 - **pre:** $m \in \mathcal{M}, k \in TKey, v \in TValue$
 - **post:** $m' \in \mathcal{M}, m' = m \cup \langle k, v \rangle, \text{add} \leftarrow v', v' \in TValue$ where

$$v' \leftarrow \begin{cases} v'', & \text{if } \exists \langle k, v'' \rangle \in m \\ 0_{TValue}, & \text{otherwise} \end{cases}$$

ADT Map - Interface IV

- **remove**(m, k)
 - **descr:** removes a pair with a given key from the map. Returns the value associated with the key, or 0_{TValue} if the key is not in the map.
 - **pre:** $m \in \mathcal{M}, k \in TKey$
 - **post:** $remove \leftarrow v, v \in TValue$, where

$$v \leftarrow \begin{cases} v', & \text{if } \exists \langle k, v' \rangle \in m \text{ and } m' \in \mathcal{M}, \\ & m' = m \setminus \langle k, v' \rangle \\ 0_{TValue}, & \text{otherwise} \end{cases}$$

- $\text{search}(m, k)$
 - **descr:** searches for the value associated with a given key in the map
 - **pre:** $m \in \mathcal{M}, k \in T\text{Key}$
 - **post:** $\text{search} \leftarrow v, v \in T\text{Value}$, where

$$v \leftarrow \begin{cases} v', & \text{if } \exists \langle k, v' \rangle \in m \\ 0_{T\text{Value}}, & \text{otherwise} \end{cases}$$

- `iterator(m, it)`
 - **descr:** returns an iterator for a map
 - **pre:** $m \in \mathcal{M}$
 - **post:** $it \in \mathcal{I}$, it is an iterator over m .
- **Obs:** The iterator for the map is similar to the iterator for other ADTs, but the *getCurrent* operation returns a $\langle \text{key}, \text{value} \rangle$ pair.

- **size(m)**
 - **descr:** returns the number of pairs from the map
 - **pre:** $m \in \mathcal{M}$
 - **post:** $\text{size} \leftarrow$ the number of pairs from m

- **isEmpty(m)**
 - **descr:** verifies if the map is empty
 - **pre:** $m \in \mathcal{M}$
 - **post:** $isEmpty \leftarrow \begin{cases} true, & \text{if } m \text{ contains no pairs} \\ false, & \text{otherwise} \end{cases}$

Other possible operations I

- Other possible operations
- $\text{keys}(m, s)$
 - **descr:** returns the set of keys from the map
 - **pre:** $m \in \mathcal{M}$
 - **post:** $s \in \mathcal{S}$, s is the set of all keys from m

Other possible operations II

- `values(m, b)`
 - **descr:** returns a bag with all the values from the map
 - **pre:** $m \in \mathcal{M}$
 - **post:** $b \in \mathcal{B}$, b is the bag of all values from m

Other possible operations III

- **pairs**(m, s)
 - **descr**: returns the set of pairs from the map
 - **pre**: $m \in \mathcal{M}$
 - **post**: $s \in \mathcal{S}$, s is the set of all pairs from m

ADT Sorted Map

- We can have a Map where we can define an order (a relation) on the set of possible keys
- The only change in the interface is for the *init* operation that will receive the *relation* as parameter.
- For a sorted map, the iterator has to iterate through the pairs in the order given by the *relation*, and the operations *keys* and *pairs* return SortedSets.

- Morse Code, is a code which assigns to every letter a sequence of dots and dashes.

A ● -	J ● - - -	S ● ● ●
B - ● ● ●	K - ● -	T -
C - ● - ●	L ● - ● ●	U ● ● -
D - ● ●	M - -	V ● ● ● -
E ●	N - ●	W ● - -
F ● ● - ●	O - - -	X - ● ● -
G - - ●	P ● - - ●	Y - ● - -
H ● ● ● ●	Q - - ● -	Z - - ● ●
I ● ●	R ● - ●	

<https://medium.com/@timboucher/learning-morse-code-35e1f4d285f6>

- Given a list of words, find the largest subset of the words, for which the Morse representation is the same.

- For example, if the words are *cat*, *ca*, *nna*, *abc* and *nnet*, their Morse code representation is:
 - *cat* -.-.-
 - *ca* -.-.-
 - *nna* -.-.-
 - *abc* .-...-.-
 - *nnet* -.-.-
- How would you solve the problem and what would be the characteristics of the container used for the solution?

- For example, if the words are *cat*, *ca*, *nna*, *abc* and *nnet*, their Morse code representation is:
 - *cat* -.-.-
 - *ca* -.-.-
 - *nna* -.-.-
 - *abc* .-...-.-
 - *nnet* -.-.-
- How would you solve the problem and what would be the characteristics of the container used for the solution?
 - We could solve the problem if we used the Morse representation of a word as a key and the corresponding word as a value
 - One key can have multiple values
 - Order of the elements is not important
- The container in which we store key - value pairs, and where a key can have multiple associated values, is called an **ADT MultiMap**.

- Domain of ADT MultiMap:

$\mathcal{MM} = \{mm \mid mm \text{ is a Multimap with TKey, TValue, pairs}\}$

ADT MultiMap - Interface I

- **init** (mm)
 - **descr:** creates a new empty multimap
 - **pre:** true
 - **post:** $mm \in \mathcal{MM}$, mm is an empty multimap

ADT MultiMap - Interface II

- `destroy(mm)`
 - **descr:** destroys a multimap
 - **pre:** $mm \in \mathcal{MM}$
 - **post:** the multimap was destroyed

ADT MultiMap - Interface III

- **add**(mm, k, v)
 - **descr:** add a new pair to the multimap
 - **pre:** $mm \in \mathcal{MM}, k - TKey, v - TValue$
 - **post:** $mm' \in \mathcal{MM}, mm' = mm \cup \langle k, v \rangle$

ADT MultiMap - Interface IV

- **remove**(mm, k, v)
 - **descr:** removes a key value pair from the multimap
 - **pre:** $mm \in \mathcal{MM}, k - TKey, v - TValue$
 - **post:** $remove \leftarrow \begin{cases} true, & \text{if } \langle k, v \rangle \in mm, mm' \in \mathcal{MM}, mm' = mm - \langle k, v \rangle \\ false, & \text{otherwise} \end{cases}$

- `search(mm, k, l)`
 - **descr:** returns a list with all the values associated to a key
 - **pre:** $mm \in \mathcal{MM}$, $k \in TKey$
 - **post:** $l \in \mathcal{L}$, l is the list of values associated to the key k . If k is not in the multimap, l is the empty list.

ADT MultiMap - Interface VI

- **iterator**(mm , it)
 - **descr:** returns an iterator over the multimap
 - **pre:** $mm \in \mathcal{MM}$
 - **post:** $it \in \mathcal{I}$, it is an iterator over mm , the current element from it is the first pair from mm , or, it is invalid if mm is empty
- **Obs:** the iterator for a MultiMap is similar to the iterator for other containers, but the *getCurrent* operation returns a $\langle \text{key}, \text{value} \rangle$ pair.

ADT MultiMap - Interface VII

- **size(mm)**
 - **descr:** returns the number of pairs from the multimap
 - **pre:** $mm \in \mathcal{MM}$
 - **post:** $size \leftarrow$ the number of pairs from mm

ADT MultiMap - Interface VIII

- Other possible operations:
- `keys(mm, s)`
 - **descr:** returns the set of all keys from the multimap
 - **pre:** $mm \in \mathcal{MM}$
 - **post:** $s \in \mathcal{S}$, s is the set of all keys from mm

- `values(mm, b)`
 - **descr:** returns the bag of all values from the multimap
 - **pre:** $mm \in \mathcal{MM}$
 - **post:** $b \in \mathcal{B}$, in b is a bag with all the values from mm

ADT MultiMap - Interface X

- `pairs(mm, b)`
 - **descr:** returns the bag of all pairs from the multimap
 - **pre:** $mm \in \mathcal{MM}$
 - **post:** $b \in \mathcal{B}$, b is a bag with all the pairs from mm

- We can have a MultiMap where we can define an order (a relation) on the set of possible keys. However, if a key has multiple values, the values can be in any order (we order the keys only, not the values) $\Rightarrow$ **ADT SortedMultiMap**
- The only change in the interface is for the *init* operation that will receive the *relation* as parameter.
- For a sorted MultiMap, the iterator has to iterate through the pairs in the order given by the *relation*, and the operations *keys* and *pairs* return SortedSet and SortedBag.

- There are several data structures that can be used to implement an ADT MultiMap (or ADT SortedMultiMap), the dynamic array being one of them (others will be discussed later):
- Regardless of the data structure used, there are two options to represent a MultiMap (sorted or not):
 - Store individual $\langle \text{key}, \text{value} \rangle$ pairs. If a key has multiple values, there will be multiple pairs containing this key (R1).
 - Store unique keys and for each key store a *list* of associated values (R2).

ADT MultiMap - R1

- For the example with the Morse code, we would have:

 cat	 ca	 nna	 abc	 nnet
---	--	---	---	--

- Key is written with red and the value with black.
- Every element is one key - value pair.

- For the example with the Morse code, we would have:

-. .-. [cat]	-. .-. [ca, nna, nnet]	.- .-. .- [abc]
--------------	------------------------	-----------------

- Key is written with red and the value with black.
- Every element is one key together with all the values belonging to it. The *list of values* can be another dynamic array, or a linked list, or any other data structure.
- Regardless of the representation, conceptually the MultiMap has key-value pairs → the iterator has to return key-value pairs (it cannot return a key and an entire list of values).



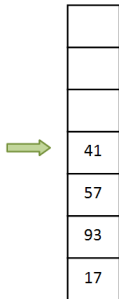
Source: <https://medium.com/analytics-vidhya/coding-interview-question-c8f2520faf72>

- If you had to take a plate, which one would you take?
- If you had to add another plate in the pile, where would you put it?

- The ADT *Stack* represents a container in which access to the elements is restricted to one end of the container, called the *top* of the stack.
 - When a new element is added, it will automatically be added to the top.
 - When an element is removed the one from the top is automatically removed.
 - Only the element from the top can be accessed.
- Because of this restricted access, the stack is said to have a **LIFO** policy: **L**ast **I**n, **F**irst **O**ut (the last element that was added will be the first element that will be removed).

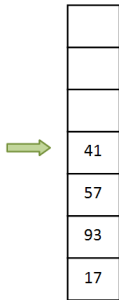
ADT Stack Example

- Suppose that we have the following stack (green arrow shows the top of the stack):
- We *push* the number 33:

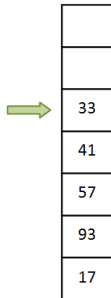


ADT Stack Example

- Suppose that we have the following stack (green arrow shows the top of the stack):



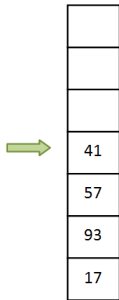
- We *push* the number 33:



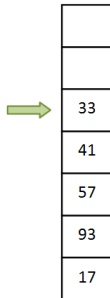
- We *pop* an element:

ADT Stack Example

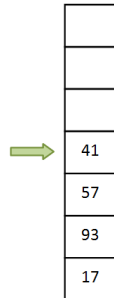
- Suppose that we have the following stack (green arrow shows the top of the stack):



- We *push* the number 33:

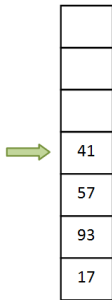


- We *pop* an element:



ADT Stack Example

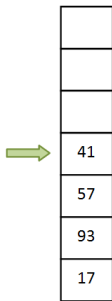
- This is our stack:



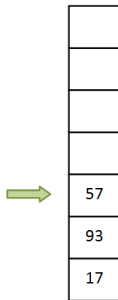
- We *pop* another element:

ADT Stack Example

- This is our stack:



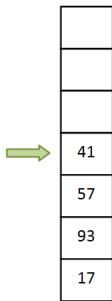
- We *pop* another element:



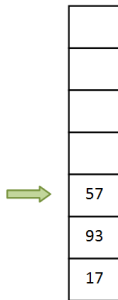
- We *push* the number 72:

ADT Stack Example

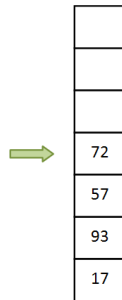
- This is our stack:



- We *pop* another element:



- We *push* the number 72:



- The domain of the ADT Stack:
 $\mathcal{S} = \{s | s \text{ is a stack with elements of type } TElem\}$
- The interface of the ADT Stack contains the following operations:

- **init(s)**
 - **descr:** creates a new empty stack
 - **pre:** True
 - **post:** $s \in \mathcal{S}$, s is an empty stack

- **destroy(s)**
 - **descr:** destroys a stack
 - **pre:** $s \in \mathcal{S}$
 - **post:** s was destroyed

- **push**(s, e)
 - **descr:** pushes (adds) a new element onto the stack
 - **pre:** $s \in \mathcal{S}$, e is a $TElem$
 - **post:** $s' \in \mathcal{S}$, $s' = s \oplus e$, e is the most recent element added to the stack

- **pop(s)**

- **descr:** pops (removes) the most recent element from the stack
- **pre:** $s \in \mathcal{S}$, s is not empty
- **post:** $pop \leftarrow e$, e is a $TElem$, e is the most recent element from s , $s' \in \mathcal{S}$, $s' = s \ominus e$
- **throws:** an *underflow* exception if the stack is empty

- **top(s)**
 - **descr:** returns the most recent element from the stack (but it does not change the stack)
 - **pre:** $s \in \mathcal{S}$, s is not empty
 - **post:** $top \leftarrow e$, e is a $TElem$, e is the most recent element from s
 - **throws:** an *underflow* exception if the stack is empty

- **isEmpty(s)**
 - **descr:** checks if the stack is empty (has no elements)
 - **pre:** $s \in \mathcal{S}$
 - **post:**

$$isEmpty \leftarrow \begin{cases} \text{true, if } s \text{ has no elements} \\ \text{false, otherwise} \end{cases}$$

- **Note:** stacks cannot be iterated, so they don't have an *iterator* operation!

- Today we have talked about:
 - ADT Bag
 - ADT Matrix - and possible representations on a dynamic array
 - ADT Set
 - ADT Map and ADT MultiMap
 - ADT SortedBag, ADT SortedSet, ADT SortedMap and ADT SortedMultiMap
 - ADT Stack